# GrdAutomationWorker — Documentação Técnica

Serviço em segundo plano (Worker) que orquestra o ciclo completo de automação N&A Connect

Resumo funcional do arquivo **GrdAutomationWorker.cs**, explicando o propósito da classe, o fluxo de execução do robô e como cada etapa do processamento se encaixa.

*Documento gerado automaticamente a partir da análise do código-fonte.*

# 1. Visão Geral

**GrdAutomationWorker** é um *BackgroundService* do ASP.NET (.NET Hosted Service) — ou seja, um processo que roda continuamente em segundo plano dentro da aplicação, sem depender de requisições HTTP de um usuário. Ele é o "maestro" do robô: a cada execução, ele consulta o banco de dados para descobrir quais projetos estão ativos e, para cada um deles, executa uma sequência de 11 etapas de sincronização entre o ACC (Autodesk Construction Cloud), o banco de dados SQL e a planilha de controle de documentos (LDP).

O arquivo também define uma classe auxiliar, **ConsoleOutputInterceptor**, usada para duplicar tudo que é escrito no console (a "tela preta") também para um arquivo de log em disco, em tempo real.

---

# 2. ConsoleOutputInterceptor

É uma classe que herda de `TextWriter` e substitui a saída padrão do console (`Console.Out`). Sempre que algo é escrito com `Console.Write`/`WriteLine`, o texto é enviado tanto para o console original quanto para um arquivo `.txt` em disco.

- Cria a pasta de destino do log automaticamente, caso não exista.
- Remove códigos de cor ANSI (usados para colorir texto no terminal) antes de gravar no arquivo, para manter o log limpo e legível.
- Usa `AutoFlush = true`, garantindo que cada linha seja gravada imediatamente em disco (útil para acompanhar a execução em tempo real ou recuperar o log em caso de falha).

# 3. GrdAutomationWorker — Estrutura

A classe herda de `BackgroundService` e recebe por injeção de dependência: o `IServiceProvider` (para criar escopos de serviço), o `ILogger` (para logs estruturados) e o `IHostApplicationLifetime` (para controlar o ciclo de vida da aplicação).

## 3.1 ExecuteAsync — ponto de entrada

É o método executado automaticamente quando a aplicação inicia. Na versão atual do código, ele está configurado em **modo de teste**: aguarda 5 segundos após o início e então dispara uma única execução completa do ciclo (`ProcessarCicloCompleto`), finalizando em seguida — ou seja, não fica repetindo em loop automaticamente; é preciso reiniciar a aplicação para rodar novamente.

# 4. ProcessarCicloCompleto — O Coração do Robô

Este método concentra toda a lógica de negócio. Para facilitar o entendimento, ele pode ser dividido em preparação, loop por projeto (com 11 etapas) e finalização.

## 4.1 Preparação

- Zera os contadores de telemetria da API Autodesk (`AutodeskApiService.ResetCounter()`).
- Inicia um cronômetro (`Stopwatch`) para medir o tempo total de execução.
- Ativa o `ConsoleOutputInterceptor`, redirecionando toda a saída do console para um arquivo de log com nome baseado em data/hora (ex.: `Execucao_Completa_20260713_143000.txt`), salvo em `C:\Dev\N-A-Connect\Logs`.

- Resolve, via injeção de dependência (dentro de um novo *scope*), todos os serviços necessários: `DatabaseService`, `AutodeskApiService`, `LdpUpdateService`, `TimesheetDumpService`, `PdfMetadataService`, `GrdToLdpSyncService`, `LdpToCustoSyncService` e `AccScheduleSyncService`.
- Consulta o banco de dados (`dbService.GetProjetosAtivos()`) para obter a lista de projetos marcados como ativos (`Is_Ativo = 1`).
- Obtém um token 2-legged (de "robô", sem usuário) junto à Autodesk, reaproveitado entre os projetos.

## 4.2 Loop por projeto — as 11 etapas

Para cada projeto ativo encontrado, o robô executa a seguinte sequência (com tratamento de erro individual por projeto, para que a falha de um não interrompa os demais):

- **Descoberta e download da LDP** — se o arquivo da LDP ainda não estiver em cache no banco, o robô localiza a pasta "09.14 - GESTÃO" (em variações PT/EN) e identifica o primeiro Excel dentro dela; em seguida baixa esse arquivo para uma pasta temporária local.
- **Etapa 0 — Validação do DNA do Projeto**: identifica o padrão de nomenclatura, idioma e demais características do projeto a partir da LDP baixada, gravando esse "DNA" no banco de dados. Se não for identificado, o processamento desse projeto é abortado por segurança.
- **Etapa 1/11 — Timesheet**: reconstrói a aba TIMESHEET da planilha via consulta SQL bruta (`ResetAndDumpTimesheetAsync`).
- **Etapa 2/11 — Horas e Revisão**: atualiza a LDP com as horas apontadas no banco e o atributo de revisão (`UpdateLdpFromDatabaseAsync`).
- **Etapa 4/11 — GRDs**: rastreia e sincroniza informações de envio de documentos (Guia de Remessa de Documentos) para a LDP (`SyncGrdsToLdpAsync`).
- **Etapa 5/11 — Workflow**: atualiza na LDP a etapa atual do fluxo de revisão do ACC (`UpdateWorkflowStatusAsync`).
- **Etapa 6/11 — Custo**: sincroniza os dados da LDP para a aba de CUSTO (`ExecuteFullSyncProcessAsync`).
- **Etapa 7/11 — Atributos ACC**: sincroniza os metadados/atributos customizados no ACC a partir do SQL (`SyncAccAttributesFromSqlAsync`). Após esta etapa o token é renovado.
- **Etapa 8/11 — Vereditos de Aprovação**: sincroniza os resultados (aprovado/reprovado/etc.) do fluxo de revisão (`RunApprovalVerdictSyncAsync`). O token é renovado novamente em seguida.
- **Etapa 9/11 — Cronograma**: sincroniza dados de cronograma do projeto para a LDP (`SyncScheduleToLdpAsync`).
- **Etapa 10/11 — Espelhamento**: copia/espelha a LDP de controle para uma pasta pública do projeto (`MirrorLdpControlToProjectAsync`).
- **Etapa 11/11 — Persistência no SQL Server**: abre o arquivo Excel local com EPPlus, percorre as linhas da aba LDP a partir da linha 8 e até 35 colunas, tratando datas e valores especiais ("-", "N/A") e grava cada linha válida na tabela unificada do SQL Server via `dbService.SalvarDadosLdpUnificada`.

## 4.3 Encerramento por projeto

- Em caso de erro em qualquer etapa, o erro é logado, mas o processamento segue para o próximo projeto (não interrompe o lote inteiro).
- No bloco `finally`, o arquivo Excel temporário baixado é sempre excluído do disco.
- Ao final de cada projeto, é logado um resumo de performance: tempo decorrido, número de chamadas feitas à API da Autodesk e o rate-limit restante.

### 4.4 Finalização do lote

Depois de processar todos os projetos, o robô imprime um resumo final (tempo total de execução, total de chamadas à API e rate-limit final) e restaura a saída do console ao seu estado original, encerrando a interceptação de log.

## 5. Métodos Auxiliares de Tratamento de Dados

### 5.1 InsinuaSerData

Função heurística que tenta adivinhar se um valor de célula do Excel representa uma data, mesmo sem tipagem explícita. Considera datas: valores já do tipo `DateTime`; números que caem numa faixa plausível de "serial date" do Excel (entre 35.000 e 60.000); ou textos que contenham "/" ou "-" no formato típico de datas.

### 5.2 TratarData

Converte, de forma "blindada" (sem lançar exceções), um valor bruto em um `DateTime` válido ou em `DBNull.Value`. Trata explicitamente marcadores comuns de campo vazio ou não aplicável ("-", "N/A", "NA", "TBD", "A DEFINIR", "S/D"), tenta o parse com cultura *pt-BR* e, como alternativa, interpreta o valor como número serial de data do Excel (`DateTime.FromOADate`), sempre validando se o ano resultante está entre 1900 e 2100 para descartar conversões absurdas.

## 6. Resumo

O **GrdAutomationWorker** é o orquestrador central da automação: um serviço de background que, a cada execução, varre os projetos ativos no banco de dados e, para cada um, executa 11 etapas encadeadas de sincronização entre o ACC, o SQL Server e a planilha LDP — cobrindo desde apontamento de horas e revisões até vereditos de aprovação, cronograma e persistência final dos dados. Ele também cuida de detalhes operacionais importantes, como renovação de token entre etapas sensíveis, isolamento de falhas por projeto, geração de logs completos em arquivo e tratamento robusto de datas vindas de planilhas Excel.

---

*Documento gerado a partir da leitura do código-fonte de GrdAutomationWorker.cs.*